CS-471: Machine Learning

Week 5 — Practical Aspects of ML: Regularisation, Cross-Validation, Feature Engineering & Introduction to Decision Trees

Instructor: Dr. Hashir Kiani Sources: Lectures 5-1 & 5-2

Comprehensive Study Notes

Week 5

Contents

1 Preventing Overfitting and Underfitting **2**
- 1.1 Strategies to Prevent Overfitting . . . 2
- 1.2 Strategies to Prevent Underfitting . . . 2

2 Using More Data & Learning Curves **3**
- 2.1 Learning Curves . . . 3
- 2.2 High-Bias Model (Underfitting) . . . 3
- 2.3 High-Variance Model (Overfitting) . . . 3

3 Cross-Validation **4**
- 3.1 Why Cross-Validation? . . . 4
- 3.2 Vanilla Cross-Validation (Hold-Out) . . . 4
- 3.3 K-Fold Cross-Validation . . . 4
- 3.4 Leave-One-Out Cross-Validation (LOOCV) . . . 5

4 Feature Selection **6**

5 Feature Engineering **6**

6 Regularisation **7**
- 6.1 Motivation . . . 7
- 6.2 The Regularised Loss Function . . . 7
- 6.3 L2 Regularisation (Ridge) . . . 8
- 6.4 L1 Regularisation (Lasso) . . . 8
- 6.5 Intuition via Housing Example . . . 8
- 6.6 Gradient Descent with L2 Regularisation . . . 9
- 6.7 Regularised Logistic Regression . . . 9
- 6.8 Choosing the Regularisation Parameter λ . . . 10

7 Early Stopping **10**

8 Decision Trees — Introduction **11**
8.1 Motivation 11
8.2 Key Properties of Decision Trees 11
8.3 Structure of a Decision Tree 12
8.4 The Golf Dataset — A Walkthrough 12
8.5 Intuitive Decision Tree for the Golf Dataset 13
8.6 Why Decision Trees Work Well for This Problem 13

9 Practice Questions **14**

10 Key Takeaways — Week 5 **15**

1 Preventing Overfitting and Underfitting

This section consolidates the key *practical* strategies to address the two fundamental failure modes of machine learning models.

1.1 Strategies to Prevent Overfitting

Recall: overfitting means the model memorises training data (high variance, low training error, high test error).

1. **Reduce model complexity** — Use a simpler hypothesis class (fewer parameters, lower polynomial degree, shallower tree).
2. **Use regularisation** — Add a penalty term to the loss function to discourage large weights (detailed in Section 6).
3. **Use cross-validation** — Choose hyperparameters/model complexity based on validation performance, not training performance (Section 3).
4. **Early stopping** — During iterative training, stop before the model begins to overfit the training data (Section 7).
5. **Use more data / data augmentation** — A more complex model needs more data to generalise; augmenting existing data (flips, rotations, noise) synthetically increases dataset size (Section 2).

1.2 Strategies to Prevent Underfitting

Recall: underfitting means the model is too simple (high bias, high training *and* test error).

1. **Increase model complexity** — Use a richer hypothesis class (deeper tree, higher-degree polynomial, more layers).
2. **Train for a longer period of time** — More gradient descent iterations may allow the model to reach a better optimum.
3. **Feature engineering** — Construct new, informative features from existing ones (Section 5).
4. **Reduce regularisation** — A very large regularisation penalty forces weights towards zero, effectively making the model too simple.
5. **Try a different ML technique** — Sometimes the model family itself is wrong; switch from linear regression to a tree-based or kernel-based method.

Key Idea

Diagnosis before treatment: Before applying any fix, first diagnose the problem using the training/validation error gap:

- High training error AND high validation error $\Rightarrow$ **underfitting** (high bias). Fix: more complexity.
- Low training error AND high validation error $\Rightarrow$ **overfitting** (high variance).

Fix: regularise or more data.

2 Using More Data & Learning Curves

2.1 Learning Curves

A **learning curve** plots training error and validation error as a function of the number of training examples N. It is a powerful diagnostic tool.

2.2 High-Bias Model (Underfitting)

Behaviour as N increases:

- Training error *rises* as N grows (harder to perfectly fit more data with a simple model).
- Validation error *falls* initially but quickly plateaus.
- The two curves converge to a *high plateau* — both errors are large and close together.

Key Idea

Getting more training data will NOT help a high-bias model.
Both curves converge early at a high error level, meaning more data does not lower the test error appreciably. The fundamental bottleneck is the model's limited capacity, not the lack of data. **Fix:** increase model complexity, add features.

Visual intuition: Imagine fitting a straight line to data that clearly curves. Whether you have 50 points or 5000 points, the straight line still cannot capture the curve — the error stays high.

2.3 High-Variance Model (Overfitting)

Behaviour as N increases:

- Training error starts near zero (overfitting on small N) and gradually rises as the model can no longer perfectly memorise all examples.
- Validation error starts very high and gradually decreases.
- There is a large **gap** between training and validation error that narrows as N grows.

Key Idea

Getting more training data IS likely to help a high-variance model.
More data forces the model to generalise rather than memorise, closing the training/validation gap. However, you may need a *very* large dataset — data augmentation can help when real data is expensive.

Visual intuition: Imagine fitting a degree-9 polynomial to 10 points (zero training error). Feed it 10,000 points and it can no longer zigzag through every point — it must approximate the true pattern, reducing overfitting.

Situation	Training error	Validation error
High bias (underfit)	High	High (similar to train)
High variance (overfit)	Low	Much higher than train
Good fit	Low	Low (close to train)

3 Cross-Validation

3.1 Why Cross-Validation?

A single train/validation split can be unreliable, especially with small datasets. A model tuned on one specific validation split might not generalise. Cross-validation averages performance over multiple splits to give a more robust estimate of generalisation error.

3.2 Vanilla Cross-Validation (Hold-Out)

1. Split dataset randomly into training set and validation set (e.g. 80%/20%).
2. Train model on training set.
3. Evaluate on validation set.

Pros: Simple and fast.
Cons: Highly sensitive to how the split is made; wastes data for validation.

3.3 K-Fold Cross-Validation

1. Randomly shuffle the dataset.
2. Divide into K equally sized **folds** (subsets).
3. For $k = 1, 2, \ldots, K$:
   (a) Use fold k as the **validation set**.
   (b) Use all remaining $K - 1$ folds as the **training set**.
   (c) Train the model and record validation error E_k.
4. Report average error: $\bar{E} = \frac{1}{K} \sum_{k=1}^{K} E_k$.

Key Idea

Every data point is used for validation *exactly once* and for training $K - 1$ times. This makes the estimate of generalisation error much more stable. Common choices: $K = 5$ or $K = 10$.

Real-Life Analogy: Imagine grading 30 students' essays. Instead of one teacher marking all 30 (biased), you form 5 groups of 6; in each round 4 groups mark the essays of the remaining group. Every student's work gets marked 4 times by different judges — the average grade is fairer. That is 5-fold cross-validation.

Example

5-Fold Cross-Validation on 100 examples:

Round	Validation fold	Training folds
1	fold 1 (examples 1–20)	folds 2,3,4,5
2	fold 2 (examples 21–40)	folds 1,3,4,5
3	fold 3 (examples 41–60)	folds 1,2,4,5
4	fold 4 (examples 61–80)	folds 1,2,3,5
5	fold 5 (examples 81–100)	folds 1,2,3,4

Each round trains on 80 examples and validates on 20. Final generalisation estimate: average of 5 validation errors.

3.4 Leave-One-Out Cross-Validation (LOOCV)

LOOCV is the extreme case of K-fold with $K = N$ (number of examples).

1. For each example $i \in \{1, \ldots, N\}$:
   (a) Train on all examples *except* example i.
   (b) Validate on example i alone.
2. Average the N validation errors.

Pros:

- Maximum use of data for training.
- Nearly unbiased estimate of true generalisation error.

Cons:

- Computationally expensive — requires training N models.
- High variance in the estimate (each validation set has only one example).
- Practical only for very small datasets.

Method	#Models trained	Best for	Computational cost
Vanilla (hold-out)	1	Large datasets	Low
K-fold ($K = 5$ or 10)	K	Small-medium datasets	Moderate
LOOCV ($K = N$)	N	Very small datasets	High

4 Feature Selection

Feature selection is the process of choosing a meaningful *subset* of the available features to use as model inputs.

Why reduce features?

- Irrelevant features add noise and can cause the model to overfit.
- Fewer features mean faster training and inference.
- Simpler models are easier to interpret.

Disadvantage: By discarding features, the algorithm throws away potentially useful information. If a discarded feature is relevant (even weakly), model performance may suffer.

Common techniques:

- **Filter methods:** rank features by statistical score (correlation, mutual information) independently of the model.
- **Wrapper methods:** evaluate subsets of features by training/validating the model on each subset (expensive but accurate).
- **Embedded methods:** let the model itself select features during training (e.g. ℓ_1 / Lasso regularisation drives some weights to exactly zero).

Common Mistake / Pitfall

Feature selection and regularisation are complementary but different. Feature selection *removes* features entirely. Regularisation *shrinks* weights towards zero but keeps all features in the model. ℓ_1 regularisation is special: it can perform *implicit* feature selection by forcing some weights to exactly zero.

5 Feature Engineering

Feature engineering is the process of creating new features from existing ones using domain knowledge, in order to better capture the relationship between inputs and outputs.

When is it needed?

- The current set of features is small or insufficient.
- The model is underfitting despite adequate complexity.
- Domain knowledge suggests that derived quantities are more informative than raw measurements.

Examples of feature engineering:

Domain	Raw features	Engineered features
Housing prices	Square footage, floors	footage^2, log(footage), price per floor
NLP	Word tokens	TF-IDF scores, n-gram frequencies
Time series	Timestamps	Hour of day, day of week, is weekend
Chemistry	Molecular weight, atoms	Bonding ratios, molecular polarity index

Key Idea

Feature engineering is one of the most impactful (and often underrated) steps in classical ML. In deep learning, neural networks learn their own features automatically — but for linear models and shallow trees, manual feature engineering can be the difference between a mediocre and an excellent model.

6 Regularisation

6.1 Motivation

Large model weights tend to produce complex, wiggly functions that overfit training data. **Regularisation** adds a **penalty term** to the loss function that discourages large weight values, effectively constraining the model's complexity without changing the hypothesis class.

6.2 The Regularised Loss Function

$$\boxed{J(\mathbf{w}) = \underbrace{\text{TrainLoss}(\mathbf{w})}_{\text{fit term}} + \underbrace{\frac{\lambda}{2N}\sum_{j=1}^{d} w_j^2}_{\text{regularisation term}}}$$

Variable definitions:

- TrainLoss($\mathbf{w}$) — the original loss (e.g. MSE or cross-entropy) measuring fit to training data.
- $\lambda \geq 0$ — **regularisation hyperparameter** (controls strength of penalty).
- w_j — weight of feature j (the bias w_0 is typically *not* regularised).
- The $1/2N$ factor is a scaling convention for cleaner gradients.

What this achieves: Minimising $J(\mathbf{w})$ simultaneously:

1. *Fits the data well* (minimises TrainLoss).
2. *Keeps weights small* (minimises $\sum w_j^2$).

Key Idea

The two-goal interpretation:
Regularisation balances:

- Goal 1 (first term): Minimise prediction error on training data.
- Goal 2 (second term): Keep model parameters small to avoid overly complex hypotheses.

λ controls the tradeoff between these two goals.

6.3 L2 Regularisation (Ridge)

The penalty added is proportional to the *square* of each weight:

$$\text{L2 penalty} = \frac{\lambda}{2N} \sum_{j=1}^{d} w_j^2 = \frac{\lambda}{2N} \|\mathbf{w}\|_2^2$$

Effect: Pushes all weights towards zero smoothly. Rarely drives any weight to exactly zero. Also called **Ridge regression** in the context of linear regression.

6.4 L1 Regularisation (Lasso)

The penalty is proportional to the *absolute value* of each weight:

$$\text{L1 penalty} = \frac{\lambda}{N} \sum_{j=1}^{d} |w_j| = \frac{\lambda}{N} \|\mathbf{w}\|_1$$

Effect: Can drive some weights to *exactly* zero, performing implicit feature selection. Also called **Lasso regression**.

Property	L2 (Ridge)	L1 (Lasso)
Penalty	$\sum w_j^2$	$\sum \lvert w_j \rvert$
Gradient	$\lambda w_j / N$ (smooth)	$\lambda \,\text{sign}(w_j)/N$ (non-smooth)
Drives weights to zero?	Approximately	Exactly
Feature selection?	No	Yes (implicit)
Works with correlated features?	Yes	May arbitrarily pick one
Typical use	When all features are relevant	When sparsity is desired

6.5 Intuition via Housing Example

Suppose we have a degree-4 polynomial for predicting house prices:

$$h(x) = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$$

The high-degree terms w_3x^3 and w_4x^4 are responsible for overfitting oscillations. Adding the penalty $+1000w_3^2 + 1000w_4^2$ to the loss forces w_3 and w_4 towards zero, effectively reducing the polynomial to a lower-degree (smoother) fit.

Example

Effect of very large λ:
If $\lambda \to \infty$: the penalty term dominates, forcing *all* $w_j \approx 0$. The model becomes $h(x) \approx w_0$ — a constant prediction (the mean of training outputs). This is severe **underfitting** (horizontal line for housing prices).
If $\lambda = 0$: no penalty, standard training loss is minimised. The model can overfit freely.
If λ is tuned well: the model is smooth and generalises well.

λ value	Model behaviour	Result
$\lambda = 0$	No regularisation	Likely overfitting (high variance)
Small λ	Mild regularisation	Slightly constrained
Well-tuned λ	Balanced	Good generalisation
Large λ	Strong regularisation	May underfit (high bias)
$\lambda \to \infty$	All weights $\to 0$	Severe underfitting

6.6 Gradient Descent with L2 Regularisation

For regularised linear regression, the update rule becomes:

$$\boxed{w_j \leftarrow w_j - \alpha \left(\frac{1}{N} \sum_{i=1}^{N} \left[h(\mathbf{x}^{(i)}) - y^{(i)} \right] x_j^{(i)} + \frac{\lambda}{N} w_j \right)}$$

Simplifying:

$$w_j \leftarrow w_j \left(1 - \frac{\alpha\lambda}{N} \right) - \frac{\alpha}{N} \sum_{i=1}^{N} \left[h(\mathbf{x}^{(i)}) - y^{(i)} \right] x_j^{(i)}$$

Key Idea

The factor $\left(1 - \frac{\alpha\lambda}{N}\right) < 1$ (for $\alpha, \lambda, N > 0$) means that *before* the usual gradient update, each weight is multiplied by a value slightly less than 1.
This is called **weight decay** — at every step, weights are shrunk (decayed) slightly towards zero, then updated with the gradient. This is why L2 regularisation is also called weight decay in deep learning frameworks.

6.7 Regularised Logistic Regression

The same L2 penalty can be added to the logistic regression training loss:

$$J(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y^{(i)}\log h(\mathbf{x}^{(i)}) + (1-y^{(i)})\log(1-h(\mathbf{x}^{(i)}))\Big] + \frac{\lambda}{2N}\sum_{j=1}^{d} w_j^2$$

The gradient descent update for w_j becomes:

$$w_j \leftarrow w_j - \frac{\alpha}{N}\sum_{i=1}^{N}\big[h(\mathbf{x}^{(i)}) - y^{(i)}\big]\, x_j^{(i)} - \frac{\alpha\lambda}{N}\, w_j$$

Identical structure to the regularised linear regression update — only the expression for $h(\mathbf{x}^{(i)})$ differs (sigmoid instead of dot product).

6.8 Choosing the Regularisation Parameter λ

λ is a **hyperparameter** — it is not learned from data during training; it must be set externally.

Standard procedure using K-fold cross-validation:

1. Choose a grid of λ values to try, e.g.: $\lambda \in \{0, 0.001, 0.01, 0.1, 1, 10, 100, 1000\}$.
2. For each candidate λ:
   (a) Train model on training folds.
   (b) Evaluate on validation fold.
3. Select the λ that achieves the lowest *cross-validated* validation error.
4. Retrain on the full training set with selected λ.
5. Report final performance on the held-out **test set**.

Common Mistake / Pitfall

Never select hyperparameters using the test set!
The test set must only be used *once* at the very end to report final performance. If you use the test set to choose λ, you are effectively overfitting to the test set and your reported performance will be optimistically biased. Always use the validation set (or cross-validation) for hyperparameter selection.

7 Early Stopping

Early stopping is a regularisation technique specific to iterative learning algorithms (e.g. gradient descent, neural network training).

Observation: As training progresses:

- Training error *monotonically decreases* (the model fits the training data better and better).

- Validation error initially decreases (the model is learning genuine patterns) but *eventually increases* once the model starts to overfit.

Strategy: Monitor validation error at each epoch (or every k epochs). Stop training when validation error stops improving and starts to rise. The optimal stopping point is the epoch at which validation error is minimised.

Key Idea

Early stopping is essentially saying: "we can see on the validation set that the model has learned enough; any further training only makes it overfit." It is computationally free (no extra parameters to tune) and surprisingly effective in practice. It is one of the most commonly used regularisation methods in deep learning.

Implementation detail: In practice, a **patience** hyperparameter p is used: stop training if the validation error has not improved in the last p consecutive epochs. This avoids premature stopping due to noisy validation error fluctuations.

Real-Life Analogy: Imagine revising for an exam by doing practice tests. Your score on unseen practice tests keeps improving for the first week. After that, you start memorising the specific questions rather than learning concepts — your score on *new* practice tests stops improving or even drops. The optimal revision time was at the peak — stopping there is early stopping.

8 Decision Trees — Introduction

8.1 Motivation

All models studied so far (linear/logistic regression, SVM-style classifiers) are **linear** — they draw straight-line (or hyperplane) decision boundaries. Many real-world problems are inherently non-linear and involve complex interactions between features.

Decision trees offer a fundamentally different, intuitive approach:

Key Idea

A **decision tree** is a flowchart-like model that makes predictions by asking a sequence of binary questions about the input features, routing each example down branches until reaching a leaf node that gives the prediction.

8.2 Key Properties of Decision Trees

- **Versatility:** Can be used for both *classification* and *regression* tasks.
- **Mixed data types:** Handle both numeric (continuous) and categorical (non-numeric) features naturally.
- **Interpretability:** Easy to visualise and explain. A doctor or manager can follow the tree logic directly — unlike neural networks ("black boxes").
- **Selective feature usage:** A decision tree only uses the features that actually matter; irrelevant features are simply not split on. This gives implicit feature selection.

- **Conditional feature relevance:** A feature might only matter under certain conditions (e.g. "Humidity matters only when Outlook is Sunny"). Decision trees model this naturally.
- **Multiple subtrees:** Complex problems may be represented by multiple trees (ensemble methods like Random Forests, Gradient Boosted Trees).
- **Goal:** Find the smallest possible tree that still classifies/predicts accurately. Smaller trees = simpler rules = better generalisation.

8.3 Structure of a Decision Tree

Component	Definition
Root node	The first split; tests the most informative feature
Internal node	Tests a feature and branches left/right (or by value)
Branch	Outcome of a test at a node
Leaf node	Terminal node; outputs a class label or value
Depth	Length of longest root-to-leaf path

8.4 The Golf Dataset — A Walkthrough

The lecture introduces decision trees using a canonical toy dataset: "Should we play golf today?"

Training Data (12 examples):

Ex.	Outlook	Temperature	Humidity	Windy	Play Golf?
1	Rainy	Hot	High	False	No
2	Rainy	Hot	High	True	No
3	Overcast	Hot	High	False	Yes
4	Sunny	Mild	High	False	Yes
5	Sunny	Cool	Normal	False	Yes
6	Sunny	Cool	Normal	True	No
7	Overcast	Cool	Normal	True	Yes
8	Rainy	Mild	High	False	No
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Sunny	Mild	High	True	No

Validation Data (2 examples):

Ex.	Outlook	Temperature	Humidity	Windy	Play Golf?
v1	Rainy	Cool	Normal	False	Yes
v2	Overcast	Hot	High	True	Yes

8.5 Intuitive Decision Tree for the Golf Dataset

Example

Building a tree by hand:
Step 1: Which feature best separates "Yes" and "No"?
Let's examine **Outlook** first:

- **Outlook = Overcast** (examples 3, 7, 11): All 3 are "Yes" ⇒ pure leaf. Predict **Yes**.
- **Outlook = Rainy** (examples 1, 2, 8, 10): No, No, No, Yes ⇒ 1 Yes, 3 No. Not pure. Need further splitting (e.g. on Windy):
  - Windy = True (ex. 2): No.
  - Windy = False (ex. 1, 8, 10): No, No, Yes — still not fully pure for ex. 10.
- **Outlook = Sunny** (examples 4, 5, 6, 9, 12): Yes, Yes, No, Yes, No ⇒ 3 Yes, 2 No. Split on Humidity:
  - Humidity = High (ex. 4, 12): Yes, No — still mixed.
  - Humidity = Normal (ex. 5, 9): Yes, Yes ⇒ pure leaf. Predict **Yes**.

Key insight from Outlook = Overcast: Observation: every overcast day in this dataset leads to playing golf. The tree routes all Overcast examples directly to "Yes" without further checks.

Validating on validation set:

- v1: Outlook = Rainy, Windy = False ⇒ further split needed; eventually predict Yes ✓
- v2: Outlook = Overcast ⇒ leaf: Yes ✓

Both validation examples classified correctly.

8.6 Why Decision Trees Work Well for This Problem

- **Features interact:** Whether Humidity matters depends on the Outlook. A linear classifier cannot model this interaction without explicit feature engineering.
- **Some features are irrelevant in some branches:** Temperature never appears in the best tree for this dataset, yet it was available as a feature.
- **Non-linear boundaries:** The decision boundary in feature space is an axis-aligned rectangular partition, impossible to represent with a single hyperplane.

Alternative Method / Trick

How is the best split chosen in practice?
At each node, the algorithm evaluates all possible splits on all features and chooses the one that maximises an **information gain** (or equivalently minimises **impurity**). Common impurity measures:

- **Entropy** (used in ID3 / C4.5 algorithms): $H = -\sum_k p_k \log_2 p_k$
- **Gini impurity** (used in CART, scikit-learn): $G = 1 - \sum_k p_k^2$
- **Information Gain** = Entropy (parent) − weighted avg. Entropy (children)

The feature/threshold with the highest information gain becomes the split at that node. This greedy approach builds the tree top-down.

9 Practice Questions

Practice Questions

Topic: Overfitting / Underfitting Strategies

Q1 (Easy): A model has training accuracy 62% and validation accuracy 60%. Which problem does it suffer from, and which two strategies from the lecture should you try first?
Answer: Both errors are high and close $\Rightarrow$ **underfitting** (high bias).
Try: (1) increase model complexity and (2) feature engineering.

Q2 (Easy): A neural network achieves 99% training accuracy but only 61% validation accuracy. Which two strategies are most directly applicable?
Answer: Large gap $\Rightarrow$ **overfitting** (high variance).
Try: (1) regularisation and (2) early stopping.

Topic: Cross-Validation

Q3 (Medium): You have 50 training examples and want a reliable estimate of generalisation error. Which cross-validation method is most appropriate, and why?
Answer: **K-fold (e.g. $K = 10$) or LOOCV.** With only 50 examples, a single hold-out split is unreliable. 10-fold trains on 45 examples each time and gives a stable average. LOOCV (50 models) is feasible at this scale and minimally wastes data.

Q4 (Exam-level): A 5-fold cross-validation on 100 examples yields fold errors: 0.22, 0.18, 0.21, 0.25, 0.14. What is the estimated generalisation error and its standard deviation?
Answer: $\bar{E} = (0.22 + 0.18 + 0.21 + 0.25 + 0.14)/5 = 1.00/5 = 0.20$
$\sigma = \sqrt{\frac{\sum(E_k - \bar{E})^2}{5}} = \sqrt{\frac{(0.02)^2+(0.02)^2+(0.01)^2+(0.05)^2+(0.06)^2}{5}} = \sqrt{\frac{0.0004+0.0004+0.0001+0.0025+0.0036}{5}} = \sqrt{0.0014} \approx 0.037$
Estimated generalisation error: 0.20 ± 0.037.

Topic: Regularisation

Q5 (Medium): The regularised training loss for linear regression is:

$$J(w_0, w_1, w_2) = \frac{1}{2N}\sum_{i=1}^{N}(h(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2N}(w_1^2 + w_2^2)$$

Write the gradient descent update rule for w_1 and w_2. Why is w_0 not regularised?
Answer:

$$w_j \leftarrow w_j\left(1 - \frac{\alpha\lambda}{N}\right) - \frac{\alpha}{N}\sum_{i=1}^{N}(h(\mathbf{x}^{(i)}) - y^{(i)})\, x_j^{(i)}, \quad j \in \{1, 2\}$$

w_0 is the bias (intercept); regularising it would prevent the model from fitting the mean of the data, which is undesirable. The bias should be free to take any value so the model's output is properly centred.

Q6 (Easy): For a regularised linear regression model, if λ is set to 10^5, what will happen to the decision boundary and why?
Answer: With $\lambda = 10^5$, the penalty term completely dominates. All weights $w_j \approx 0$ for $j \geq 1$. The model becomes $h(x) \approx w_0$, predicting a constant for all inputs. This is severe **underfitting** — a horizontal line for regression, or a single-class predictor for classification.

Topic: Decision Trees

Q7 (Medium): Using the golf training data, consider examples with Outlook = Sunny (examples 4, 5, 6, 9, 12). There are 5 examples: 3 "Yes", 2 "No".
(a) Compute the entropy of this subset.
(b) If we split on Humidity, the High group has {4:Yes, 12:No} and Normal group has {5:Yes, 9:Yes, 6:No}. Compute the entropy of each group and the information gain.
Solution:
(a) $p_{\text{Yes}} = 3/5$, $p_{\text{No}} = 2/5$.

$$H(\text{Sunny}) = -\tfrac{3}{5}\log_2\tfrac{3}{5} - \tfrac{2}{5}\log_2\tfrac{2}{5} \approx 0.971 - 0.529 = 0.971 \text{ bits}$$

(more precisely: $0.442 + 0.529 = 0.971$ bits)
(b) High ($n = 2$): 1 Yes, 1 No $\Rightarrow H_{\text{High}} = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$
Normal ($n = 3$): 2 Yes, 1 No $\Rightarrow H_{\text{Normal}} = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} \approx 0.918$ bits
Weighted entropy after split: $\frac{2}{5}(1.0) + \frac{3}{5}(0.918) = 0.4 + 0.551 = 0.951$
Information Gain $= 0.971 - 0.951 = 0.020$ bits.

Q8 (Easy): Why is a smaller decision tree generally preferred over a larger one, even if the larger tree achieves lower training error?
Answer: A smaller tree is simpler and makes fewer specific tests. It generalises better to unseen data because it captures underlying patterns rather than memorising training-set-specific details. Larger trees are prone to **overfitting** — they may follow noise in the training data. Occam's Razor: prefer the simplest hypothesis that fits the data adequately.

10 Key Takeaways — Week 5

Quick Summary

- **Overfitting fixes:** reduce complexity, regularise, cross-validate, early stop, more data / augmentation.
- **Underfitting fixes:** more complexity, more training, feature engineering, re-

duce regularisation, try a different model.

- **Learning curves:** high-bias model does NOT benefit from more data (both errors plateau high); high-variance model DOES benefit as more data closes the train/val gap.
- **Vanilla CV:** single train/val split — simple but noisy.
- **K-fold CV:** K splits, average validation error; every example used for val once. Preferred for small/medium datasets.
- **LOOCV:** $K = N$; near-unbiased but computationally expensive; for very small datasets.
- **Feature selection:** discard irrelevant features; may lose information.
- **Feature engineering:** create new informative features; especially powerful for underfitting models.
- **L2 regularisation (Ridge):** penalty $\lambda \sum w_j^2/2N$; shrinks weights smoothly; manifests as *weight decay*.
- **L1 regularisation (Lasso):** penalty $\lambda \sum |w_j|/N$; drives some weights to exactly zero; implicit feature selection.
- **Lambda λ:** $\lambda = 0$ (overfit), large λ (underfit); tune by cross-validation on validation set only — never on test set.
- **Early stopping:** stop iterative training at the epoch of minimum validation loss; free regularisation.
- **Decision trees:** flowchart of attribute tests; handle numeric and categorical data; interpretable; implicitly select features; prefer smallest tree.
- **Golf dataset:** Outlook $\rightarrow$ Overcast always Yes; Rainy/Sunny need further splits on Windy/Humidity.
- **Split criterion:** choose feature/threshold that maximises Information Gain (minimises weighted entropy / Gini impurity after split).